PROJECT DOCUMENTATION · REVISION B

Smart Medicine Alarm

An ESP32 pill dispenser that turns a pill disk, sounds an alarm at each dose time, and keeps ringing until a sensor confirms the tablets were taken.

Three doses a day

Dose times are set from a phone or laptop and saved in the ESP32's flash memory, so they survive a power cut.

Works without internet

The ESP32 creates its own Wi-Fi network and serves a dashboard at http://192.168.4.1.

Confirmed pick-up

An IR sensor must see the tablets and then see them removed before the alarm stops.

Gentle dispensing

The servo turns the pill disk 60° per dose with a slow start and slow stop, so tablets do not spill.

Controller	ESP32 DevKit V1 (ESP32-WROOM-32)
Firmware	`firmware/SmartMedicineAlarm/SmartMedicineAlarm.ino`
Desktop app	`dashboard/dashboard.py` (Python, CustomTkinter)
Repository	github.com/Rutu-Raaj/Smart_Medicine_Alarm
Schematic	Revision B (buzzer on GPIO 33, servo on GPIO 32)
Date	12 November 2026

This document describes the hardware, wiring, firmware logic, dashboards, device API, setup, daily use, configuration and troubleshooting of the system.

Contents

1. **Overview**
    1. What the system does
    2. A dose, step by step
    3. System architecture
    4. Repository layout
2. **Hardware**
    1. Parts list
    2. Pin map
    3. Power and wiring rules
    4. Circuit schematic
3. **Firmware**
    1. Main loop
    2. Code structure
    3. Dose cycle
    4. Alarm and tablet sensor
    5. Servo motion
    6. Clock and time sync
    7. A day on the device
    8. Safety rules
    9. LCD screens
4. **Dashboards**
    1. Built-in web dashboard
    2. Desktop dashboard
5. **Device API**
6. **Setup**
    1. Flashing the firmware
    2. Desktop dashboard
7. **Daily use**
8. **Configuration**
9. **Troubleshooting**
    1. Symptoms and fixes
    2. Testing the tablet sensor
10. **Known limitations**
11. **Change log**

1 Overview

1.1 What the system does

The Smart Medicine Alarm reminds a patient to take their medicine three times a day. Tablets for each dose sit in a compartment of a pill disk. At each dose time the ESP32 turns the disk to the next compartment, sounds a buzzer, and shows a prompt on an LCD. The alarm does not stop on a timer: it keeps sounding until an infrared (IR) sensor confirms that the tablets arrived in the tray and were then taken out.

The ESP32 creates its own Wi-Fi network, so the system needs no router and no internet. Any phone or laptop that joins this network can open the built-in dashboard, set the dose times, watch the device status and reset the day. A Python desktop dashboard offers the same functions on a laptop.

Doses per day	Default times	Disk step per dose	Wi-Fi network
3	**08:00 · 14:00 · 20:00**	**60°**	**Smart_Medicine_Alarm**

1.2 A dose, step by step

1. The device clock reaches a dose time that has not been given yet today.
2. The servo turns the pill disk 60° to the next compartment, slowly, so the tablets drop into the tray without spilling.
3. The buzzer beeps (200 ms on, 200 ms off), the LCD shows `Take Medicine`, and both dashboards show a red *Time to take medicine* alert.
4. After the disk stops, the IR sensor must see the tablets in the tray for at least 0.5 s.
5. When the patient takes the tablets and the tray stays empty for 0.5 s, the alarm stops and the LCD shows `Medicine Taken`.

1.3 System architecture

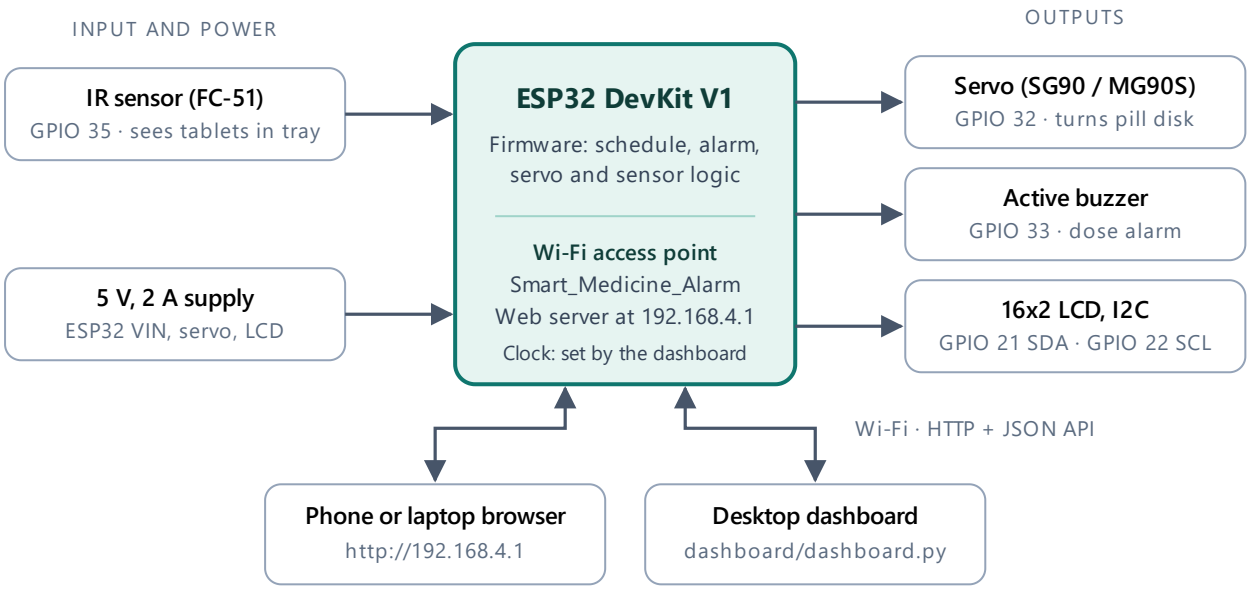


Figure 1. System architecture. The ESP32 reads the IR sensor, drives the servo, buzzer and LCD, and serves the dashboards over its own Wi-Fi network.

1.4 Repository layout

```
Smart_Medicine_Alarm/
├── firmware/
│   └── SmartMedicineAlarm/
│       └── SmartMedicineAlarm.ino   ESP32 firmware, including the built-in web dashboard
├── dashboard/
│   ├── dashboard.py                 Desktop dashboard (Python, CustomTkinter)
│   └── requirements.txt             Python dependency for the desktop dashboard
├── docs/
│   └── circuit-schematic.png        Full circuit schematic
├── .gitignore
└── README.md
```

The Arduino IDE requires a sketch to sit in a folder with the same name, which is why the firmware is in `firmware/SmartMedicineAlarm/`.

2 Hardware

2.1 Parts list

Ref	Part	Model / value	Notes
U1	ESP32 development board	DevKit V1, ESP32-WROOM-32, 30 pins	Any ESP32 board with the pins below works.
M1	Servo motor, 180°	SG90 or MG90S	Must be a positional servo, not a 360° continuous one.
BZ1	Active buzzer	3.3 V to 5 V	An active buzzer beeps on DC. A passive one only clicks.
U2	IR obstacle sensor	FC-51 (LM393 comparator)	Points at the tray. Output LOW when it sees an object.
U3	16x2 LCD with I2C backpack	LCD1602 + PCF8574	Address 0x27 (some modules use 0x3F).
C1	Electrolytic capacitor	470 to 1000 µF, 10 V or higher	Across the servo power pins, close to the servo.
PS1	DC power supply	5 V, 2 A	Or 2× 18650 cells through a buck converter set to 5 V.
–	Breadboard and jumper wires	–	Keep the grounds of all parts connected.

2.2 Pin map

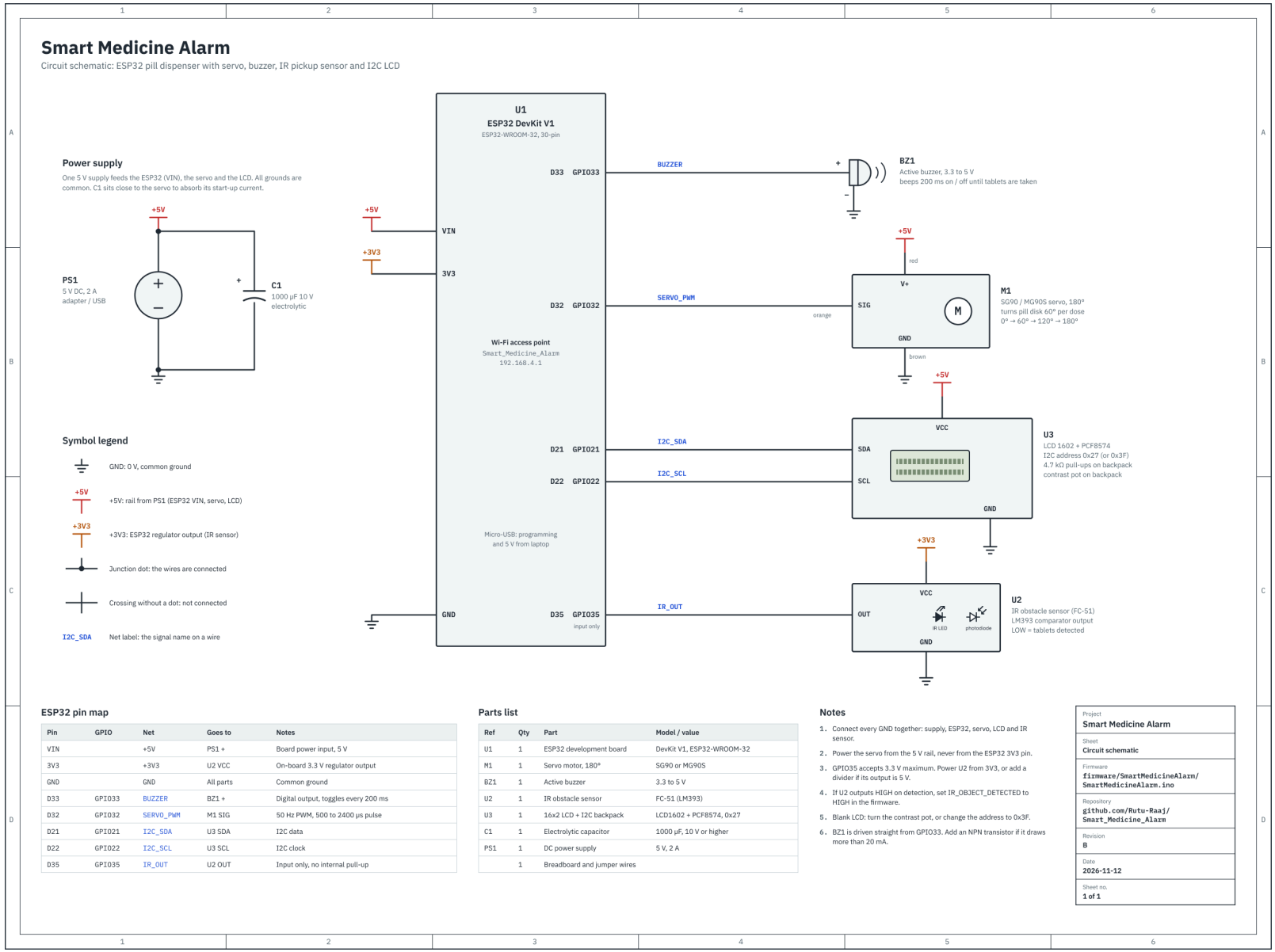
Board pin	GPIO	Signal	Connected to	Notes
D33	33	BUZZER	BZ1 (+)	Digital output, toggles every 200 ms during an alarm.
D32	32	SERVO_PWM	M1 signal (orange)	50 Hz PWM, 500 to 2400 µs pulse.
D21	21	I2C_SDA	U3 SDA	I2C data for the LCD.
D22	22	I2C_SCL	U3 SCL	I2C clock for the LCD.
D35	35	IR_OUT	U2 OUT	Input only, no internal pull-up. 3.3 V maximum.
VIN	–	+5 V	PS1 (+)	Board power input.
3V3	–	+3.3 V	U2 VCC	On-board regulator output for the IR sensor.
GND	–	GND	All parts	Common ground.

> **Why GPIO 32 and 33?**
> Revision B moved the buzzer from GPIO 18 to GPIO 33 and the servo from GPIO 19 to GPIO 32. Both new pins are plain output pins with no boot-time behaviour. Avoid GPIO 34 to 39 (input only), GPIO 0, 2, 5, 12 and 15 (boot strapping pins), GPIO 14 (pulses at boot), GPIO 1 and 3 (USB serial) and GPIO 6 to 11 (flash memory).

2.3 Power and wiring rules

- **Power the servo from 5 V, never from the ESP32's 3V3 pin.** A servo draws 650 to 700 mA when it starts moving. A weak supply, such as a laptop USB port, makes the voltage dip and resets the ESP32. The firmware detects this and shows `Power dip reset` on the LCD.
- **Use one common ground** for the supply, ESP32, servo, LCD and IR sensor.
- **Fit the 1000 µF capacitor** across the servo's power pins to absorb its start-up current.
- **Battery option:** two 18650 cells in series give 7.4 to 8.4 V. Set the buck converter output to 5 V with a multimeter before connecting the servo and LCD.
- **IR sensor output:** GPIO 35 accepts 3.3 V at most. Power the sensor from 3V3, or add a voltage divider if its output is 5 V.
- **IR polarity:** the firmware expects LOW when tablets are present. If your module works the other way, set `IR_OBJECT_DETECTED` to `HIGH`.
- **Buzzer current:** GPIO 33 drives the buzzer directly. If the buzzer draws more than 20 mA, switch it through an NPN transistor.
- **Breadboard:** the two halves of a breadboard are not connected across the centre gap. Wires that must join have to sit in the same 5-hole column.

2.4 Circuit schematic



Pin	GPIO	Net	Goes to	Notes
VIN		+5V	PS1 +	Board power input, 5 V
3V3		+3V3	U2 VCC	On-board 3.3 V regulator output
GND		GND	All parts	Common ground
D33	GPIO33	BUZZER	BZ1 +	Digital output, toggles every 200 ms
D32	GPIO32	SERVO_PWM	M1 SIG	50 Hz PWM, 500 to 2400 µs pulse
D21	GPIO21	I2C_SDA	U3 SDA	I2C data
D22	GPIO22	I2C_SCL	U3 SCL	I2C clock
D35	GPIO35	IR_OUT	U2 OUT	Input only, no internal pull-up

Ref	Qty	Part	Model / value
U1	1	ESP32 development board	DevKit V1, ESP32-WROOM-32
M1	1	Servo motor, 180°	SG90 or MG90S
BZ1	1	Active buzzer	3.3 to 5 V
U2	1	IR obstacle sensor	FC-51 (LM393)
U3	1	16x2 LCD + I2C backpack	LCD1602 + PCF8574, 0x27
C1	1	Electrolytic capacitor	1000 µF, 10 V or higher
PS1	1	DC power supply	5 V, 2 A
	1	Breadboard and jumper wires	

Figure 2. Full circuit schematic, revision B, with pin map, parts list and wiring notes.

3 Firmware

The firmware is a single Arduino sketch for the ESP32. It uses the ESP32 Arduino core (WiFi, WebServer, Preferences) and two libraries: **ESP32Servo** (version 3.x) and **LiquidCrystal I2C**. The build for the ESP32 Dev Module uses about 76% of program storage (1,001,261 of 1,310,720 bytes) and 15% of dynamic memory.

3.1 Main loop

The program never blocks. Each pass of `loop()` does a small amount of work and returns, so the web server, alarm and servo all stay responsive at the same time.

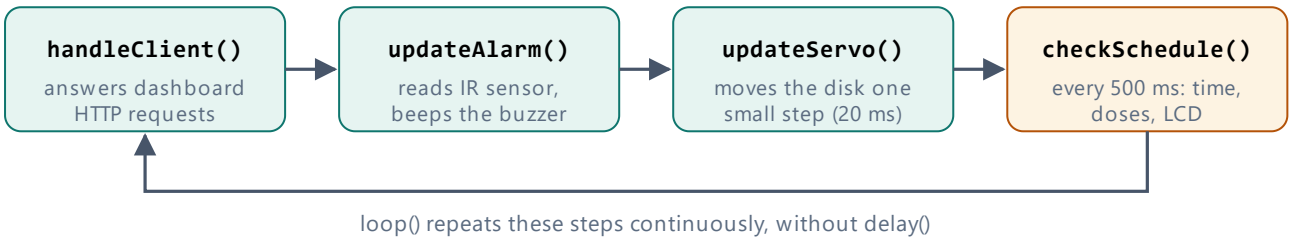


Figure 3. One pass of the main loop.

3.2 Code structure

Function	Role
`setup()`	Starts serial output, the buzzer pin, the LCD and the IR input. Loads the saved dose times, attaches the servo at 0°, starts the Wi-Fi access point and the web server. Reports a brownout reset on the LCD.
`checkSchedule()`	Runs every 500 ms. Starts a new day when the date changes, fires a dose when its minute arrives, updates the time and next-dose text, and refreshes the LCD.
`dispenseDose()`	Marks the dose as given, lowers the slot count, moves the disk to the next compartment and starts the alarm.
`moveToNextSlot()`, `updateServo()`	Set the new disk angle (+60°, at most 180°) and move the servo there smoothly in 20 ms steps.
`startAlarm()`, `updateAlarm()`	Sound the buzzer and stop it only after the tablets are seen and then removed.
`readTabletPresent()`	Filters the IR sensor: a reading counts only after it has held for `IR_STABLE_MS` (500 ms).
`resetDoses()`, `resetDayIfNeeded()`	Manual reset (all doses pending, alarm off, disk back to 0°) and the automatic new-day reset at midnight.
`saveDoseTimes()`, `loadDoseTimes()`	Store and read the three dose times in flash (Preferences namespace `med`, keys `d1h` to `d3m`).

Function	Role
`handleSetTime()`, `handleSetSchedule()`, `handleReset()`, `statusJson()`	The HTTP API used by both dashboards (section 5).

3.3 Dose cycle

A dose fires only during its exact minute, only if it has not been given yet today, and only once per minute. Figure 4 shows the full decision path from the schedule check to the end of the alarm.

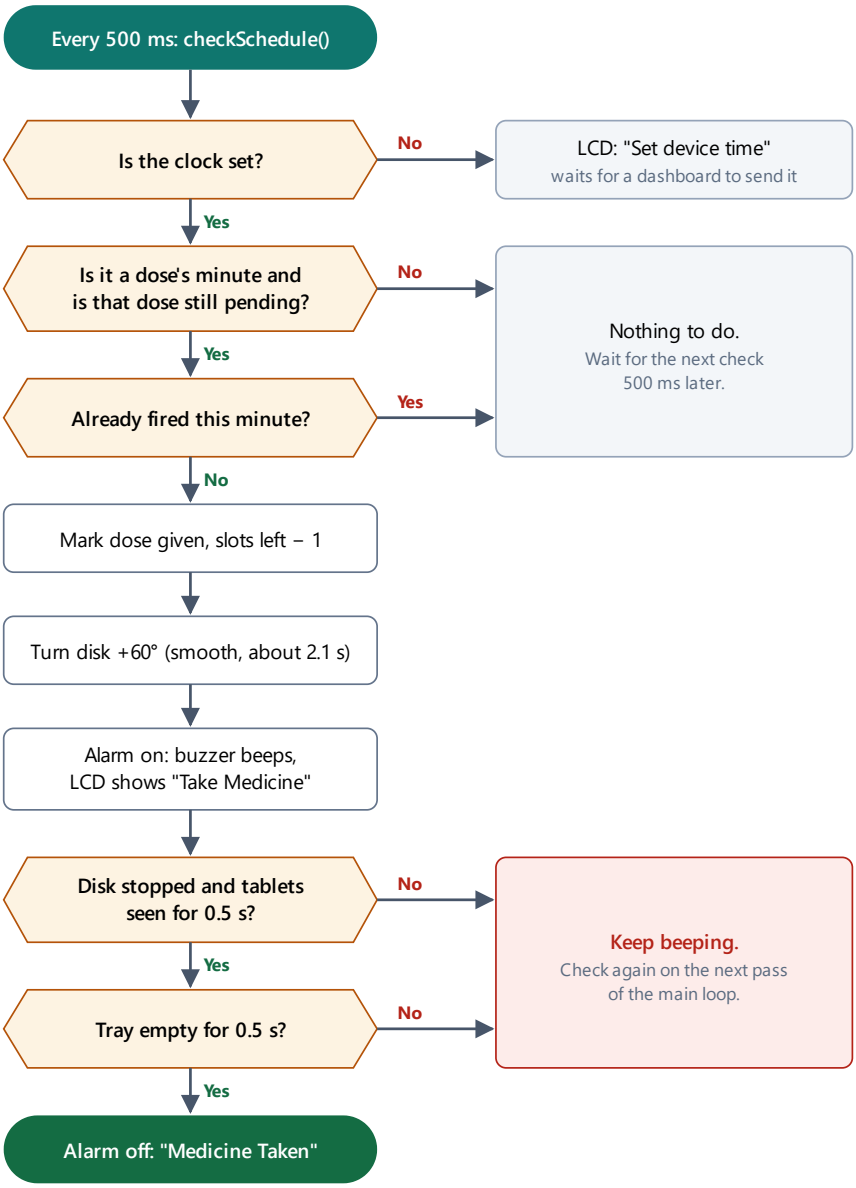


Figure 4. Dose cycle. Amber shapes are decisions, the red box is the alarm waiting state.

3.4 Alarm and tablet sensor

The alarm is a small state machine (Figure 5). It starts when a dose fires and ends only when the IR sensor has seen the tablets and then seen them removed. *Reset today's doses* on a dashboard also stops it.

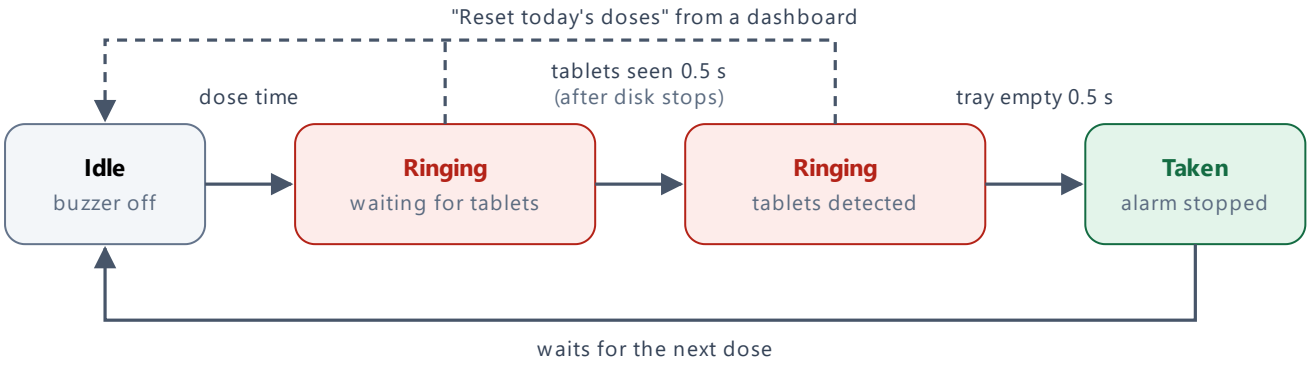


Figure 5. Alarm states. Solid arrows are normal transitions, dashed arrows are the manual reset.

How the sensor reading is filtered

- **Stable readings only.** A raw reading must stay the same for `IR_STABLE_MS` (500 ms) before the alarm trusts it. A tablet falling past the sensor, or electrical noise from the servo motor, produces short flickers that are ignored.
- **Sensor ignored while the disk turns.** The alarm does not look at the sensor until the servo has reached its new angle. The buzzer still sounds during the move.
- **Presence first, then removal.** The alarm stops only after the tablets have been seen and then removed. It never assumes they were picked up.
- **Live reading on the dashboards.** The *Tablet sensor* row always shows *Sees tablets* or *Clear*, which makes the sensor easy to test (section 9.2).

3.5 Servo motion

The disk starts at 0° and moves 60° per dose: 0° → 60° → 120° → 180°. Each move takes 35 ms per degree, so one dose step takes about 2.1 s and the return from 180° to 0° takes about 6.3 s. The position follows a cosine ease curve, `eased = (1 - cos(πt)) / 2`, which starts and stops slowly so tablets are not thrown out of their compartments. The servo is driven in microseconds (500 to 2400 µs for 0° to 180°), which gives about 0.1° resolution.

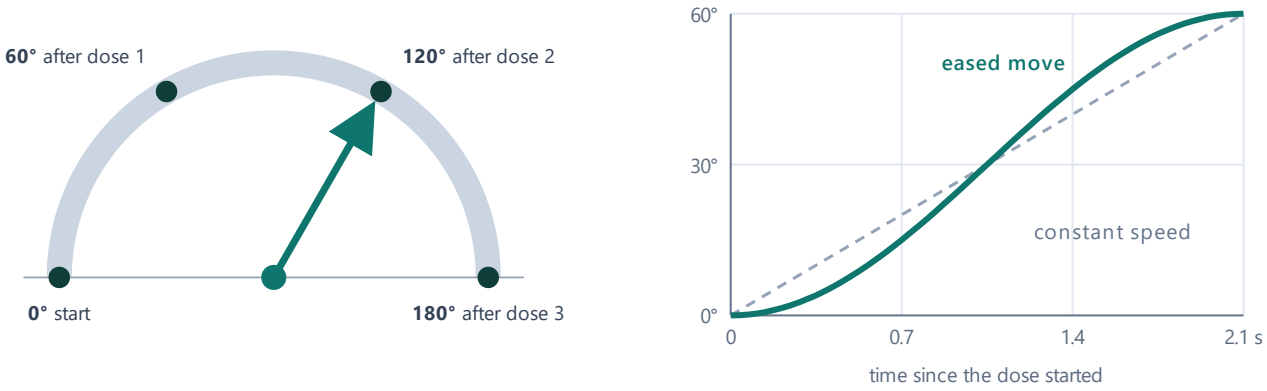


Figure 6. Left: pill disk positions through the day. Right: angle over time for one 60° step, compared with a constant-speed move.

3.6 Clock and time sync

The system has no real-time clock module and no internet time source. Instead, the dashboard sends the phone's or laptop's clock to the ESP32 as a Unix timestamp. The firmware applies India Standard Time (`IST-5:30`, UTC+5:30) to it. The clock counts as valid only for timestamps after November 2023 (epoch 1,700,000,000). After a power cut the clock is unknown again, and no dose fires until a dashboard is opened.

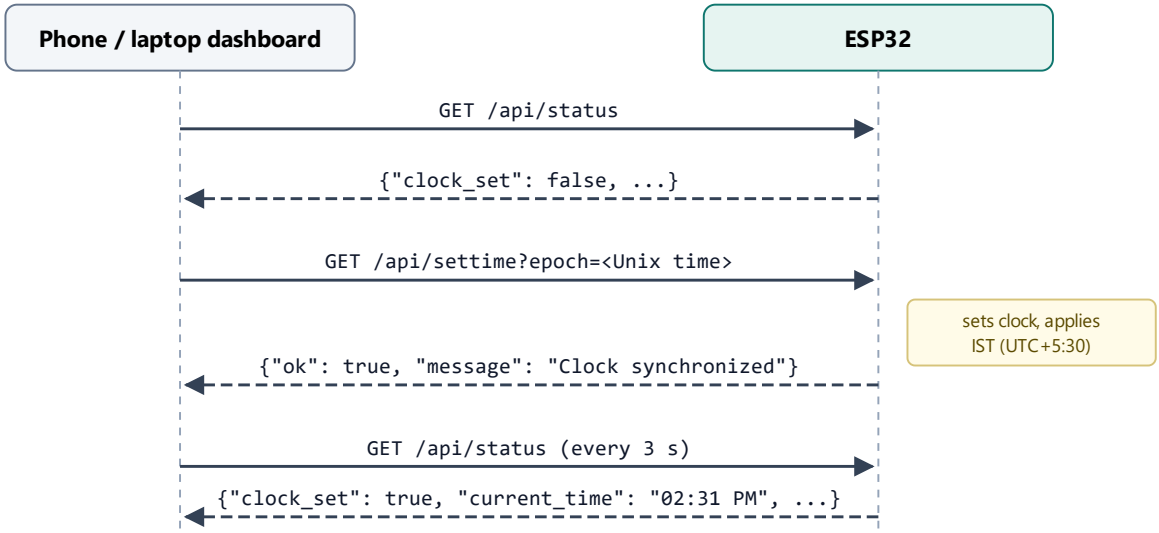
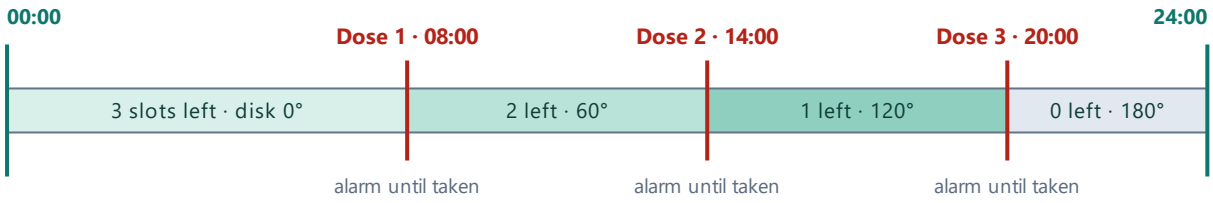


Figure 7. Clock sync. The dashboard sends its time whenever the device reports that its clock is not set, and again every hour to correct drift.

3.7 A day on the device



At midnight all doses become pending and the slot count returns to 3.
The disk stays at 180°: refill it and select **Reset today's doses**.

Figure 8. A day with the default schedule. Each dose turns the disk 60° and lowers the slot count by one.

3.8 Safety rules built into the firmware

The firmware is designed so that a dose is never dispensed twice.

- **Saving a schedule changes only the times.**
  Doses already given today stay given, the slot count is kept and the disk does not move. Changing the time of a dose that was already given does not make it fire again.
- **Same-minute guard.**
  The firmware remembers the dose, day and minute of the last dose it fired. A reset during that minute does not fire the same dose again.
- **Physical confirmation.**
  The alarm stops only when the sensor sees the tablets and then sees them removed, or when someone presses *Reset today's doses*.
- **Visible power problems.**
  If a voltage dip resets the ESP32, the LCD shows `Power dip reset` for 3 s at start-up and the serial output prints `BROWNOUT`.
- **Disk limit.**
  The disk never turns past 180°. If a dose fires while the disk is already at 180°, the serial output asks for a refill and a reset.

3.9 LCD screens

`Medicine System` `Starting...`	**Start-up** Shown for a moment after power-on.	`WiFi: Medicine` `192.168.4.1`	**Wi-Fi ready** The access point is running.
`Set device time` `Open 192.168.4.1`	**Clock not set** Open a dashboard to send the time.	`02:31 PM` `L:1 N:08:00 PM`	**Normal** Current time, slots left (L) and next dose (N).
`Take Medicine` `Remove tablets`	**Dose due** Buzzer sounding, disk turned.	`Medicine Taken` `Alarm stopped`	**Taken** The sensor saw the tablets removed.
`Reset Done` `3 slots left`	**Reset** After *Reset today's doses*.	`Power dip reset` `Check servo 5V`	**Brownout** The supply dipped and reset the ESP32.

If the access point cannot start, the LCD shows `WiFi AP failed / Restart device`.

4 Dashboards

4.1 Built-in web dashboard

The ESP32 serves its own dashboard at `http://192.168.4.1`. It is a single self-contained page with no external files, because the access point has no internet. It adapts to phone and laptop screens and follows the device's light or dark theme.

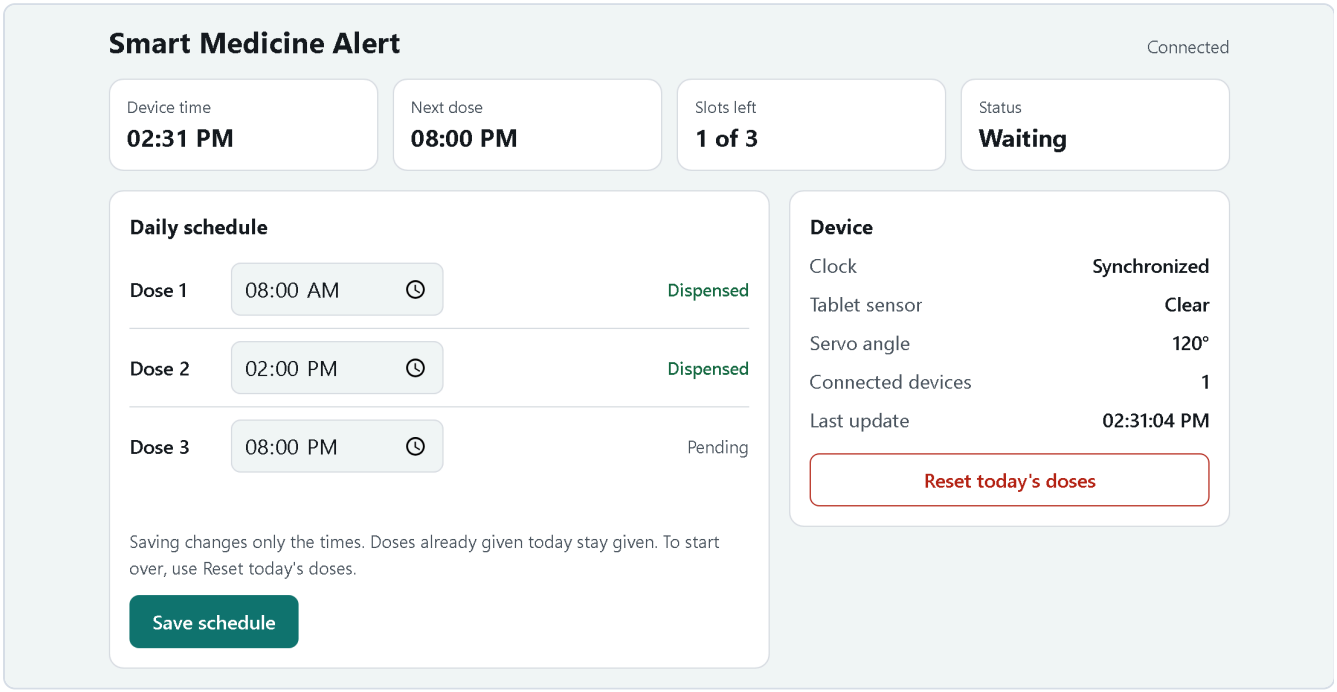


Figure 9. Web dashboard on a laptop: device time, next dose, slots left, status, daily schedule and device details.

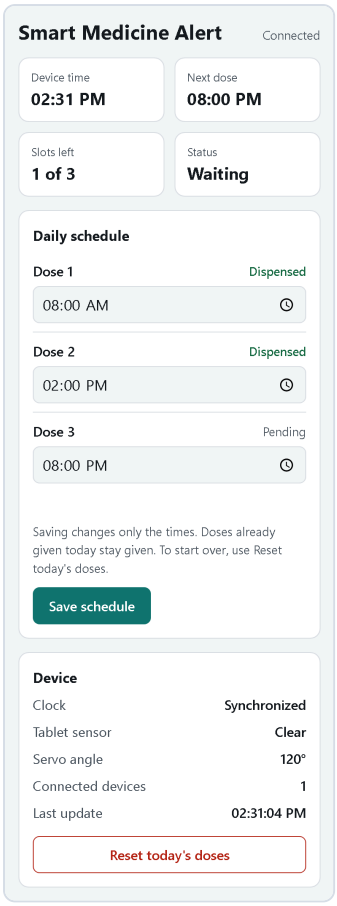


Figure 10. On a phone.

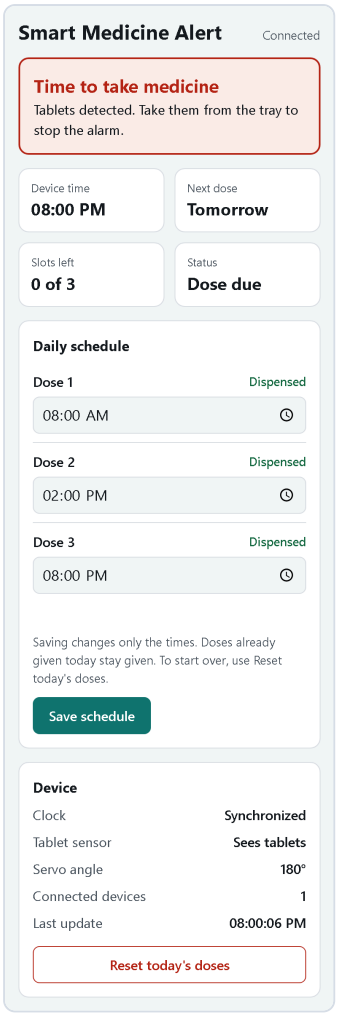


Figure 11. During an alarm.

What the page does

- Refreshes the status every 3 seconds.
- Sends the browser's clock when the device clock is not set, and again every hour.
- Shows a red *Time to take medicine* banner during an alarm and vibrates the phone when the alarm starts.
- Marks each dose as *Dispensed* or *Pending*.
- Checks that all three times are filled in and different before saving.
- Does not overwrite times you are editing while it refreshes.
- Asks for confirmation before *Reset today's doses*.
- Shows *Not connected* if the phone leaves the device's Wi-Fi.

4.2 Desktop dashboard

`dashboard/dashboard.py` is a Windows desktop app written in Python with CustomTkinter. It talks to the same device API as the web page and is useful when a laptop stays next to the dispenser.

Area	Function
Header and status bar	Connection state (Connected or Offline) and the time of the last update.
Overview	Large device clock, next dose and slots left. During an alarm a red banner replaces it, the app beeps and the window comes to the front.
Pill tray	One box per dose, coloured by state: *Take now*, *Dispensed*, *Next*, *Missed* or *Upcoming*.
Daily schedule	Hour and minute pickers in 24-hour time, *Save schedule* and *Reset doses*. Edits stay marked as unsaved until the device confirms them.
Device panel	Status, clock state with a *Sync* button, live tablet sensor reading, servo angle, connected devices and the device address.

The app refreshes every 5 seconds, sends the laptop's time at start-up and every hour, and runs all network requests on a background thread so the window never freezes. It follows the Windows light or dark theme.

5 Device API

All endpoints use HTTP GET and return JSON. Every response carries `Access-Control-Allow-Origin: *` and `Cache-Control: no-store`. The device is at `http://192.168.4.1`.

Endpoint	Parameters	Purpose
`/`	none	The built-in web dashboard (HTML).
`/api/status`	none	Current time, next dose, slots left, alarm and sensor state, servo angle and the three doses.
`/api/settime`	`epoch` (Unix time, UTC)	Sets the device clock.
`/api/schedule`	`h1, m1, h2, m2, h3, m3`	Saves three dose times (24-hour). Doses already given today stay given.
`/api/reset`	none	Marks all doses pending, stops the alarm and returns the servo to 0°.

Example: set the schedule

```
http://192.168.4.1/api/schedule?h1=8&m1=0&h2=14&m2=0&h3=20&m3=0

{"ok":true,"message":"Schedule saved"}
```

Example: status response

```
{
  "device": "Smart Medicine Alert",
  "ssid": "Smart_Medicine_Alarm",
  "ip": "192.168.4.1",
  "clock_set": true,
  "current_time": "02:31 PM",
  "next_dose": "08:00 PM",
  "status": "Waiting",
  "slots_left": 1,
  "alarm_active": false,
  "tablet_seen": false,        // tablets seen during the current alarm
  "tablet_now": false,         // live, filtered sensor reading
  "client_count": 1,
  "servo_angle": 120,
  "doses": [
    {"hour": 8,  "minute": 0, "time": "08:00 AM", "done": true},
    {"hour": 14, "minute": 0, "time": "02:00 PM", "done": true},
    {"hour": 20, "minute": 0, "time": "08:00 PM", "done": false}
  ]
}
```

Errors

Code	Response	When
400	`{"ok":false,"error":"Missing epoch"}`	`/api/settime` without `epoch`.
400	`{"ok":false,"error":"Invalid epoch"}`	Timestamp before November 2023.
400	`{"ok":false,"error":"Provide h1,m1,h2,m2,h3,m3"}`	A schedule parameter is missing.
400	`{"ok":false,"error":"Invalid time range"}`	Hour outside 0 to 23 or minute outside 0 to 59.
404	`{"ok":false,"error":"Not found"}`	Unknown path.

6 Setup

6.1 Flashing the firmware

1. Install the Arduino IDE.
2. Add the ESP32 board package. In **File > Preferences**, add this URL to *Additional boards manager URLs*:
   `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
   Then open **Tools > Board > Boards Manager**, search for *esp32* and install **esp32 by Espressif Systems**.
3. Install the libraries from **Sketch > Include Library > Manage Libraries**: **ESP32Servo** by Kevin Harrington (version 3.x) and **LiquidCrystal I2C** by Frank de Brabander.
4. Open `firmware/SmartMedicineAlarm/SmartMedicineAlarm.ino`.
5. Select **Tools > Board > esp32 > ESP32 Dev Module** and the board's port under **Tools > Port**.
6. Click **Upload**. If the upload stops at `Connecting....`, hold the board's **BOOT** button until it starts.
7. After the upload, the LCD shows `WiFi: Medicine` and `192.168.4.1`.

> **Erasing old data**
> Uploading a sketch replaces the old program, but the saved dose times stay in flash. To clear everything, set
> **Tools > Erase All Flash Before Sketch Upload**
> to *Enabled* for one upload, then set it back to *Disabled*. The dose times return to 08:00, 14:00 and 20:00.

The Arduino IDE may warn that LiquidCrystal I2C "claims to run on avr architecture". The library works on the ESP32 and the warning can be ignored.

6.2 Desktop dashboard

Tested with Python 3.11 and CustomTkinter 6.0. Connect the laptop to the **Smart_Medicine_Alarm** Wi-Fi network first.

```
cd dashboard
pip install -r requirements.txt
python dashboard.py
```

7 Daily use

First-time setup

1. Power on the dispenser and wait for `WiFi: Medicine` on the LCD.
2. On a phone or laptop, join the Wi-Fi network **Smart_Medicine_Alarm** (password `12345678`). If the phone warns that the network has no internet, choose to stay connected.
3. Open `http://192.168.4.1`. Type `http://` in full, because some browsers try `https` and fail.
4. The page sends the phone's time automatically. Check that **Device time** matches the phone.
5. Enter the three dose times and select **Save schedule**.
6. Fill the disk compartments and select **Reset today's doses**. The disk moves to its start position (0°).

Every day

- At each dose time the disk turns, the buzzer sounds and the LCD shows `Take Medicine`. Take the tablets from the tray. The alarm stops on its own about half a second later.
- After the third dose of the day, refill the disk and select **Reset today's doses**. The disk returns to 0° for the next day.
- After a power cut, open the dashboard once so the device gets the time again. The LCD shows `Set device time` until then.

Changing dose times

Edit the times on a dashboard and select **Save schedule**. Saving changes only the times: doses already given today stay given, and the disk does not move. To start the day over (for example, when testing), select **Reset today's doses**.

Testing a dose quickly

1. Select **Reset today's doses**. The disk returns to 0° and all doses become pending.
2. Set dose 1 to one or two minutes from now and save.
3. At the dose time the disk turns 60° and the buzzer starts. Make sure a tablet is in the tray, in front of the sensor.
4. Remove the tablet. After half a second the alarm stops and dose 1 shows *Dispensed*.

8 Configuration

These constants are at the top of `SmartMedicineAlarm.ino`. Change them, then upload the firmware again.

Constant	Default	Meaning
`AP_SSID`, `AP_PASSWORD`	`Smart_Medicine_Alarm`, `12345678`	Wi-Fi network created by the ESP32. Change the password before real use.
`BUZZER_PIN`	`33`	Buzzer output pin.
`SERVO_PIN`	`32`	Servo signal pin.
`IR_SENSOR_PIN`	`35`	IR sensor input pin.
`IR_OBJECT_DETECTED`	`LOW`	Sensor output level when tablets are present.
`IR_STABLE_MS`	`500`	How long a sensor reading must hold before the alarm trusts it. Raise it if the alarm stops by itself.
`SERVO_STEP_DEGREES`	`60`	Rotation per dose.
`SERVO_MAX_ANGLE`	`180`	Largest disk angle.
`SERVO_MS_PER_DEGREE`	`35`	Servo speed. A 60° step takes about 2.1 s. Larger is slower.
`SERVO_MIN_US`, `SERVO_MAX_US`	`500`, `2400`	Pulse width at 0° and 180°. Calibrate for your servo.
Timezone	`IST-5:30`	Set in `setDeviceEpoch()`. Change it for other regions.
LCD address	`0x27`	In `LiquidCrystal_I2C lcd(0x27, 16, 2)`. Some modules use `0x3F`.

In `dashboard/dashboard.py`, `REFRESH_MS` sets how often the desktop dashboard checks the device (default 5000 ms) and `RESYNC_MS` how often it re-sends the laptop's time (default one hour).

9 Troubleshooting

9.1 Symptoms and fixes

Symptom	Likely cause	Fix
LCD shows `Set device time`	The clock is unknown after power-on.	Open `http://192.168.4.1` or the desktop dashboard. The time is sent automatically.
LCD shows `Power dip reset`	The servo pulled the supply voltage down and reset the ESP32.	Use a 5 V, 2 A supply instead of a laptop USB port, fit the 1000 µF capacitor and connect all grounds.
LCD blank or shows only blocks	Contrast set wrong, or a different I2C address.	Turn the contrast screw on the backpack. Try address `0x3F`.
A dose did not fire	The clock was not set at that minute, or the dose was already given today.	Check **Device time** on the dashboard. Select **Reset today's doses** to make all doses pending again.
Dose fires at the wrong time	Device time differs from local time.	Compare **Device time** with the phone. Check the timezone in `setDeviceEpoch()`.
Servo does not move at a dose	The disk is already at 180° (day rolled over without a reset), or a wiring or power fault.	Refill the disk and select **Reset today's doses**. Check the signal wire on D32 and the 5 V supply.
Servo jitters or overshoots	Weak supply or uncalibrated pulse range.	Improve the supply. Adjust `SERVO_MIN_US` and `SERVO_MAX_US`.
Alarm stops by itself	The sensor flickers or sees the disk.	Raise `IR_STABLE_MS` to 1000. Turn down the sensor's sensitivity screw.
Alarm never stops after taking the tablets	The sensor never saw the tablets, or always sees something.	Test the sensor (section 9.2). Adjust its position and sensitivity, or check `IR_OBJECT_DETECTED`.
Buzzer only clicks	It is a passive buzzer.	Use an active buzzer.
Dashboard does not open	Not on the device's Wi-Fi, or the browser tried `https`.	Join **Smart_Medicine_Alarm**, stay connected without internet, and type `http://192.168.4.1` in full.
Upload stops at `Connecting....`	The board is not in download mode.	Hold the **BOOT** button until the upload starts. Close the Serial Monitor first.

9.2 Testing the tablet sensor

1. Open the dashboard and find the **Tablet sensor** row.
2. Hold a hand or a tablet in front of the sensor. The row should change to *Sees tablets* within about half a second, and the module's own LED should light.
3. Move it away. The row should change back to *Clear*.

Result	Meaning and fix
Changes correctly	The sensor and wiring work. Adjust the sensor's position so it sees the tablets in the tray.
Module LED lights, row stays *Clear*	The signal does not reach the ESP32. Check the OUT wire to D35 and the common ground.
Always *Sees tablets*	The sensor sees the tray or disk. Turn the sensitivity screw down or angle the sensor.
Shows the opposite of reality	The module's output is inverted. Set `IR_OBJECT_DETECTED` to `HIGH`.

10 Known limitations

- **No real-time clock module.** After a power cut the ESP32 does not know the time until a phone or laptop opens a dashboard. Doses do not fire while the clock is not set.
- **Doses fire only during their exact minute.** If the device is off or its clock is not set at that minute, the dose is skipped. The desktop dashboard shows it as *Missed*.
- **The disk is not returned to 0° at midnight.** Select **Reset today's doses** after refilling the disk each day.
- **The alarm has no timeout.** If the sensor never sees tablets (for example, an empty compartment), the buzzer keeps sounding until **Reset today's doses** is selected.
- **No authentication.** Anyone who joins the Wi-Fi network can change the schedule. Change the default Wi-Fi password.
- **Fixed timezone.** The firmware uses India Standard Time. Other regions need a code change.

11 Change log

Revision	Changes
A	First release: ESP32 firmware with built-in web dashboard and JSON API, desktop dashboard, README and circuit schematic. Buzzer on GPIO 18, servo on GPIO 19.
B 12 Nov 2026	• Saving a schedule now changes only the dose times. A dose that was just given no longer fires again when the schedule is saved during its minute. • A reset no longer makes the current minute's dose fire again. • The IR sensor reading must hold for 0.5 s and is ignored while the disk turns, so the alarm no longer stops by itself. • Both dashboards show the live tablet sensor reading. • A brownout reset is reported on the LCD and serial output. • Buzzer moved to GPIO 33 and servo to GPIO 32. README and schematic updated.